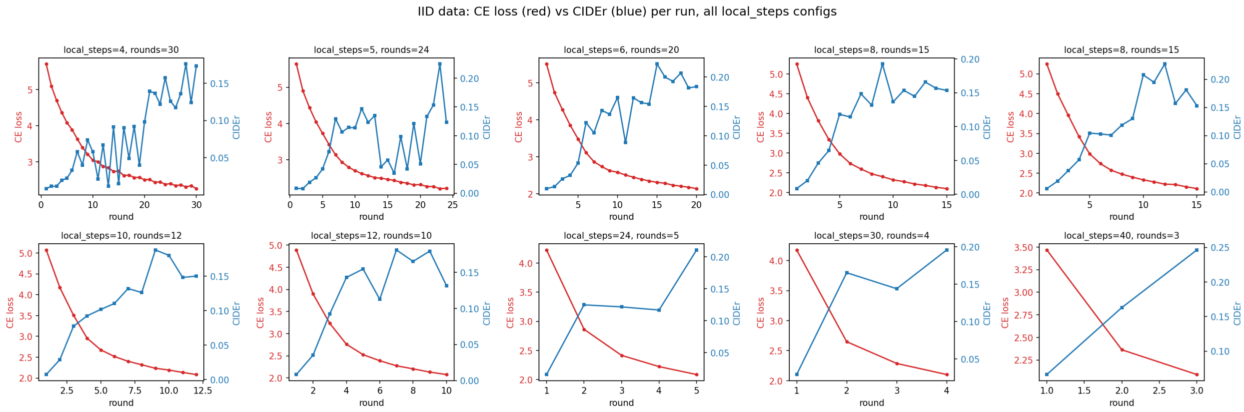
Federated Learning: Local Steps vs. Aggregation Rounds

Effect of aggregation frequency and data skew on Florence-2 captioning quality (CIDEr)

Comparing 10 runs on near-IID client data (Output_final_results) against 10 runs on skewed client data (Output_2_final_results). Every run except two holds the total local-training budget fixed at local_steps × rounds = 120, so results can be compared fairly across different aggregation frequencies; two runs on the skewed split were deliberately extended further to study convergence and overfitting.

1. All IID Runs at a Glance: CE Loss vs. CIDEr per Run



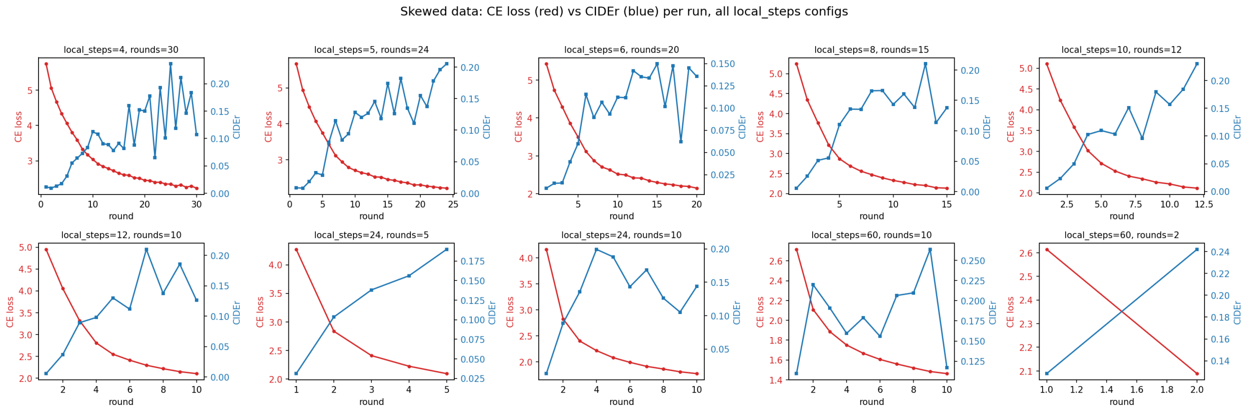
Red = aggregated cross-entropy loss (left axis). Blue = CIDEr (right axis). Each subplot is one run, labeled by its local_steps/rounds configuration.

Observation: In every configuration, CE loss falls smoothly and monotonically round over round, while CIDEr rises but with visibly more round-to-round noise, especially at low local_steps (4-8), where the CIDEr line zig-zags sharply instead of climbing cleanly.

Inference: Loss is a much better-behaved training signal than CIDEr in this setup. CIDEr is computed against a downstream generation metric (n-gram overlap of generated captions with references) rather than being the directly optimized objective, so it inherits more variance from decoding/generation randomness round to round. This means loss curves alone are not a reliable proxy for caption quality progress - you need to track CIDEr (or another generation metric) directly rather than assuming lower loss implies better captions.

Practical takeaway: When picking a checkpoint to deploy, never pick 'the round with lowest loss' - pick the round with best CIDEr, since the two curves visibly diverge in behavior, most obviously in the local_steps=24, 30, and 40 panels where CIDEr keeps rising even as loss flattens out.

2. All Skewed Runs at a Glance: CE Loss vs. CIDEr per Run



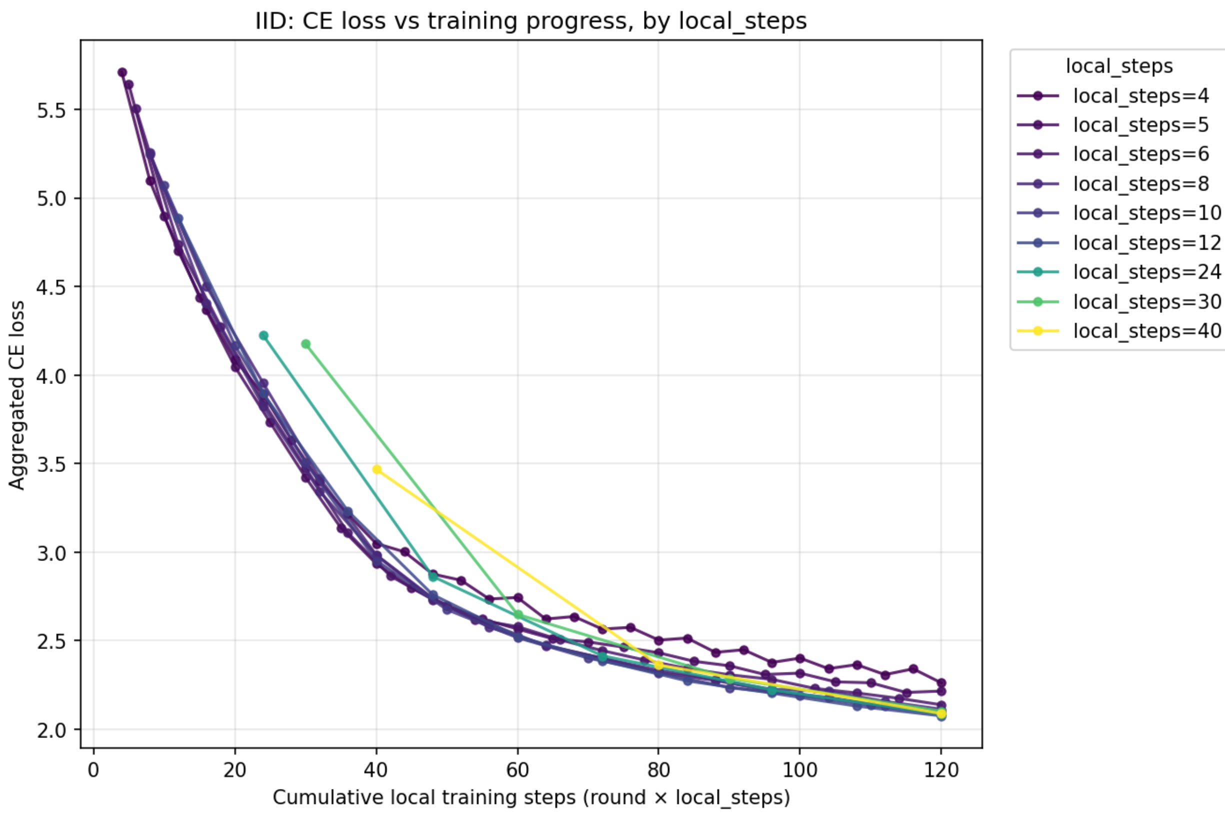
Same layout as Section 1, but for the skewed (non-IID) client data split.

Observation: The same general loss/CIDEr shape holds under skew - loss falls smoothly, CIDEr climbs noisily - but the noise in CIDEr is visibly larger here than in the IID grid, particularly for local_steps=6 and local_steps=24, which show sharp dips mid-training before recovering.

Inference: Data skew adds an extra source of instability on top of the usual generation-metric noise. When clients hold non-IID data, each client's local update pulls the shared model toward its own local distribution; the more skewed the split, the more those pulls can disagree with each other round to round, which shows up as sharper CIDEr fluctuations even while the aggregated loss keeps improving smoothly (loss is dominated by the average, so it hides this disagreement much better than CIDEr does).

Practical takeaway: Under skew, evaluate more frequently (favor lower local_steps / more rounds) if you need to reliably catch the true best checkpoint - the skewed local_steps=24 and local_steps=60 panels later in this report only get 5 and 10 evaluation points respectively, which is a thin sample to trust a single 'best round' from.

3. IID: CE Loss vs. Cumulative Training Steps, All local_steps Overlaid



X-axis is cumulative local training steps (round × local_steps), not round number, so runs with different local_steps are compared on equal footing. Color encodes local_steps (dark purple = low/frequent aggregation, yellow = high/infrequent aggregation).

Observation: All local_steps values collapse onto essentially the same loss-vs-steps curve. The family of lines overlaps tightly rather than fanning out by color.

Inference: Under IID data, aggregation frequency has almost no effect on how efficiently the loss decreases per unit of local training - a client step is worth about the same amount of loss reduction whether you sync every 4 steps or every 40. This makes sense: when every client sees a similar data distribution, frequent syncing mostly re-confirms an already-shared direction rather than correcting real disagreement between clients.

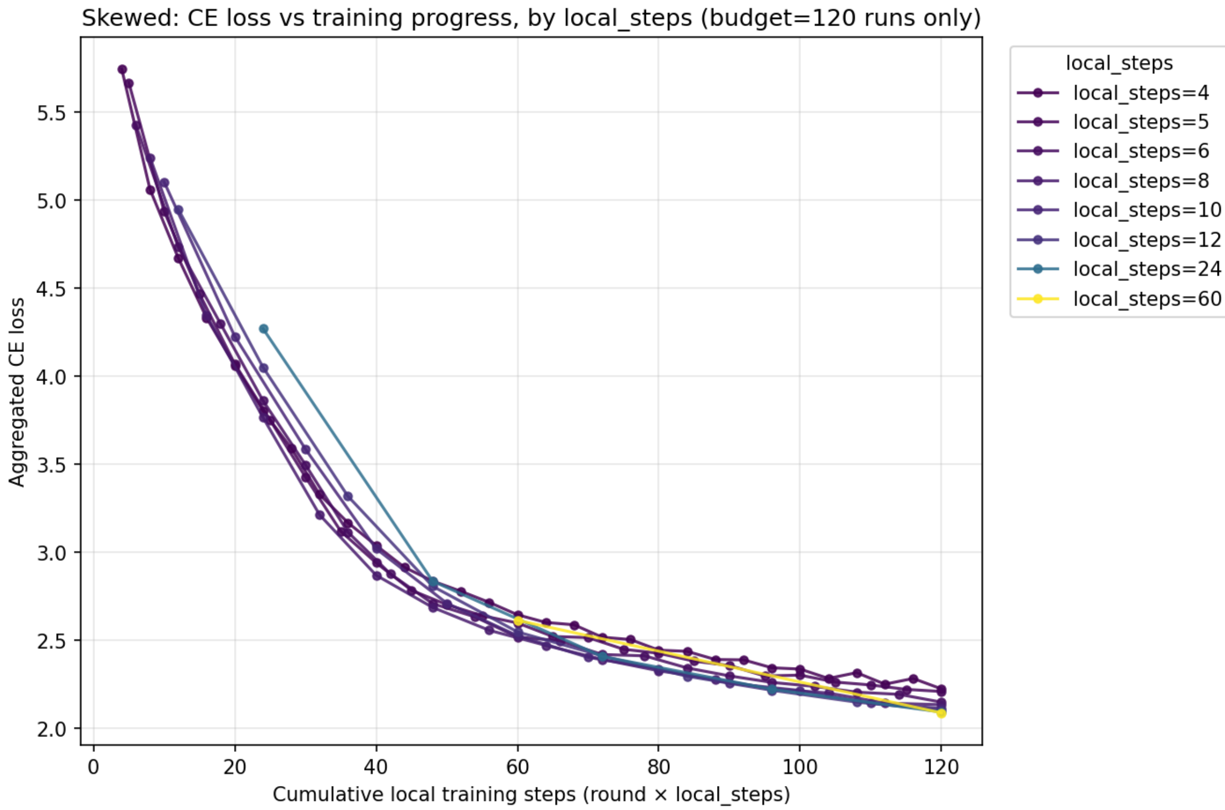
4. IID: CIDEr vs. Cumulative Training Steps, All local_steps Overlaid

Same x-axis and color scheme as Section 3, now for CIDEr.

Observation: Unlike the loss plot, the CIDEr lines do NOT collapse onto one curve. Low local_steps (dark purple, 4-8) form a wide, jagged, overlapping noisy band. High local_steps (green/yellow, 24-40) trace much smoother, almost straight-line paths from the origin to the top-right, and the local_steps=40 line reaches the single highest CIDEr point on the whole plot (~0.246) at the full 120-step budget.

Inference: This is the clearest visual evidence in the IID data that infrequent aggregation produces steadier apparent progress - but with only 3 rounds total for local_steps=40, that 'smoothness' is also just fewer samples with less chance to show noise. It would be premature to call local_steps=40 the winner from this alone; Section 10 checks this claim against the skewed data and against repeat runs.

5. Skewed: CE Loss vs. Cumulative Training Steps, All local_steps Overlaid

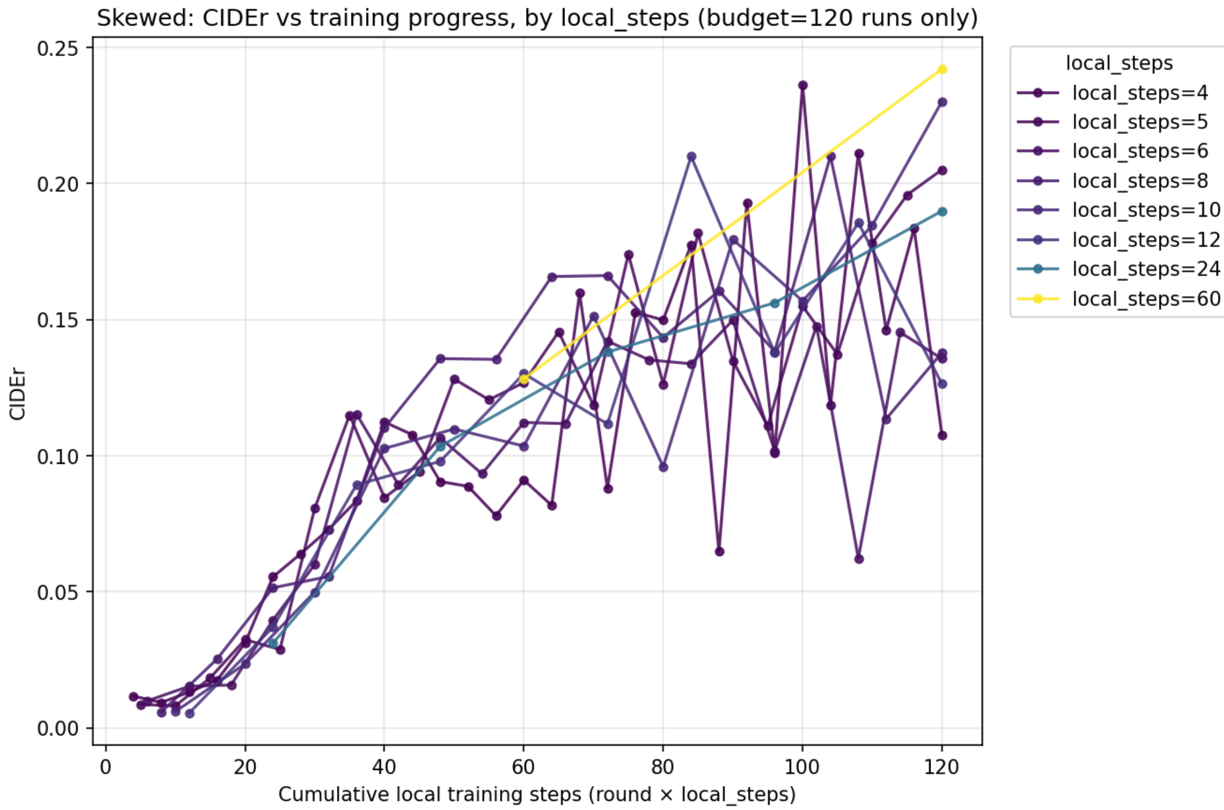


Same construction as Section 3, restricted to the 8 skewed runs that share the 120-step budget (the 2 deliberately extended runs are excluded here and analyzed separately in Section 9).

Observation: As with the IID case, loss curves for all local_steps values again collapse onto a single shared trajectory, with only slightly more spread than the IID version.

Inference: Aggregation frequency's effect on raw loss reduction efficiency is small even under data skew - the CE loss objective is fairly insensitive to how often clients synchronize. This reinforces that loss alone would not have surfaced any of the aggregation-frequency effects this report is investigating; those effects only appear in the downstream CIDEr metric and in the client-divergence numbers (Section 7).

6. Skewed: CIDEr vs. Cumulative Training Steps, All local_steps Overlaid

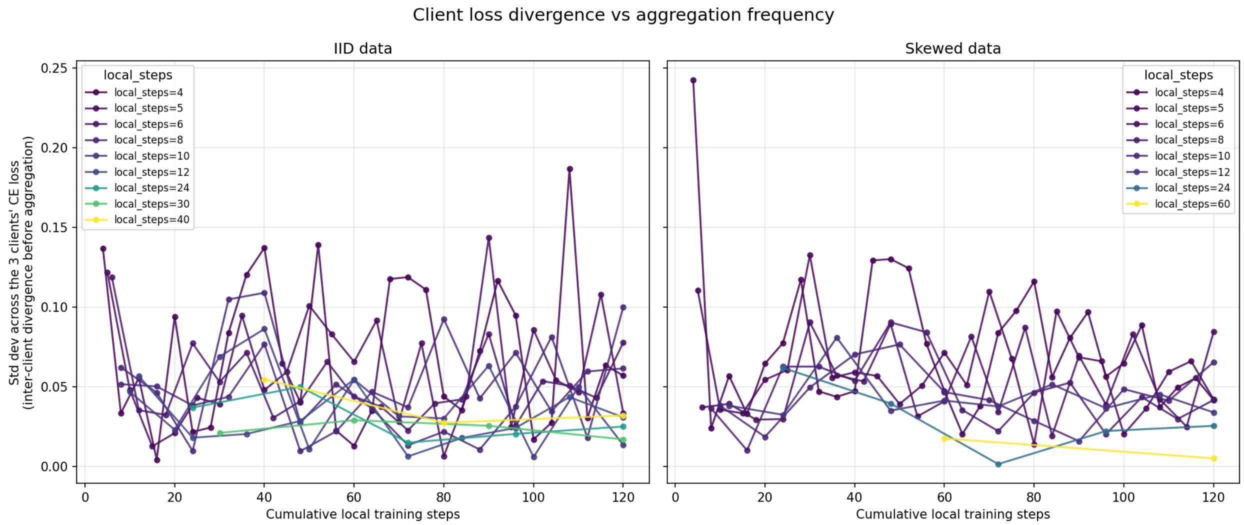


Same construction as Section 4, skewed data.

Observation: The spread between local_steps values is even wider here than in the IID CIDEr plot. Some mid-range local_steps runs (6, 24) end up well below the pack by the 120-step mark, while local_steps=4 and local_steps=10 trace some of the higher paths.

Inference: Under skew, the ranking of local_steps values by final CIDEr is noticeably reshuffled compared to IID (compare with Section 4) - the same local_steps value does not reliably transfer its ranking from one data distribution to the other. This is the first strong sign that 'best local_steps' is not a fixed constant but depends on how skewed the client data is, a point Section 10 quantifies directly.

7. Client Loss Divergence vs. Aggregation Frequency



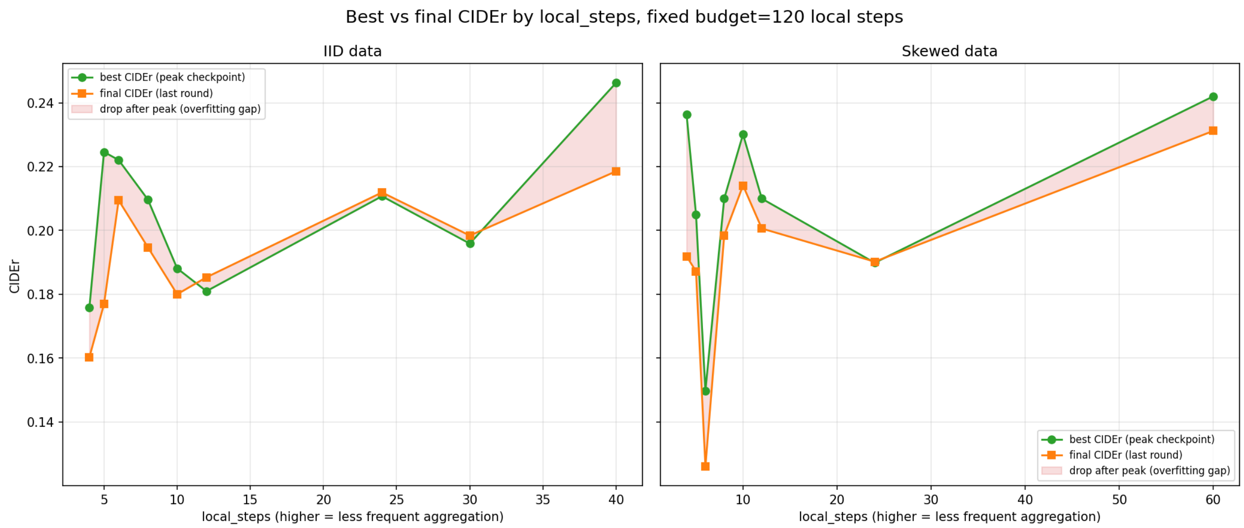
Y-axis: standard deviation across the 3 clients' CE loss at each round, i.e. how far apart the clients have drifted since the last aggregation. Left: IID. Right: Skewed.

Observation: Contrary to the intuitive expectation, divergence is consistently LOWER at higher local_steps in both panels, not higher - both the raw standard deviation shown here and the relative version (std divided by mean client loss) fall steadily as local_steps increases, from roughly 0.033 (relative) at local_steps=4 down to about 0.010-0.012 at local_steps=24-30 in IID, and down to about 0.004 at local_steps=60 in the skewed data.

Inference: This data does not support 'infrequent aggregation increases client drift' as a clean, isolated effect - if anything the opposite pattern shows up here. However this comparison is confounded: because the total budget is fixed at 120 steps, high-local_steps runs have far fewer rounds, so even their first measured round already reflects a large amount of local training (round 1 of local_steps=40 already means 40 local steps have happened, versus round 1 of local_steps=4 meaning only 4). Loss - and its spread across clients - shrinks naturally as training progresses (mean client loss falls from about 8.5 near the start to 3-5 by the high-local_steps runs' first measurements), so part or all of the apparent 'less divergence at high local_steps' may simply be an artifact of measuring later in the effective optimization process, not evidence that infrequent aggregation itself suppresses client drift.

Practical takeaway: This std-based proxy, as measured here, cannot cleanly isolate the causal effect of aggregation frequency on client divergence, because local_steps and 'how far into training the measurement is taken' are entangled by the fixed-budget design. A cleaner test would compare client_loss_std across different local_steps configs at matched cumulative_steps checkpoints (not just at round boundaries), or hold the number of rounds fixed while varying local_steps instead of holding the total budget fixed.

8. Best vs. Final CIDEr by local_steps (Fixed 120-Step Budget)



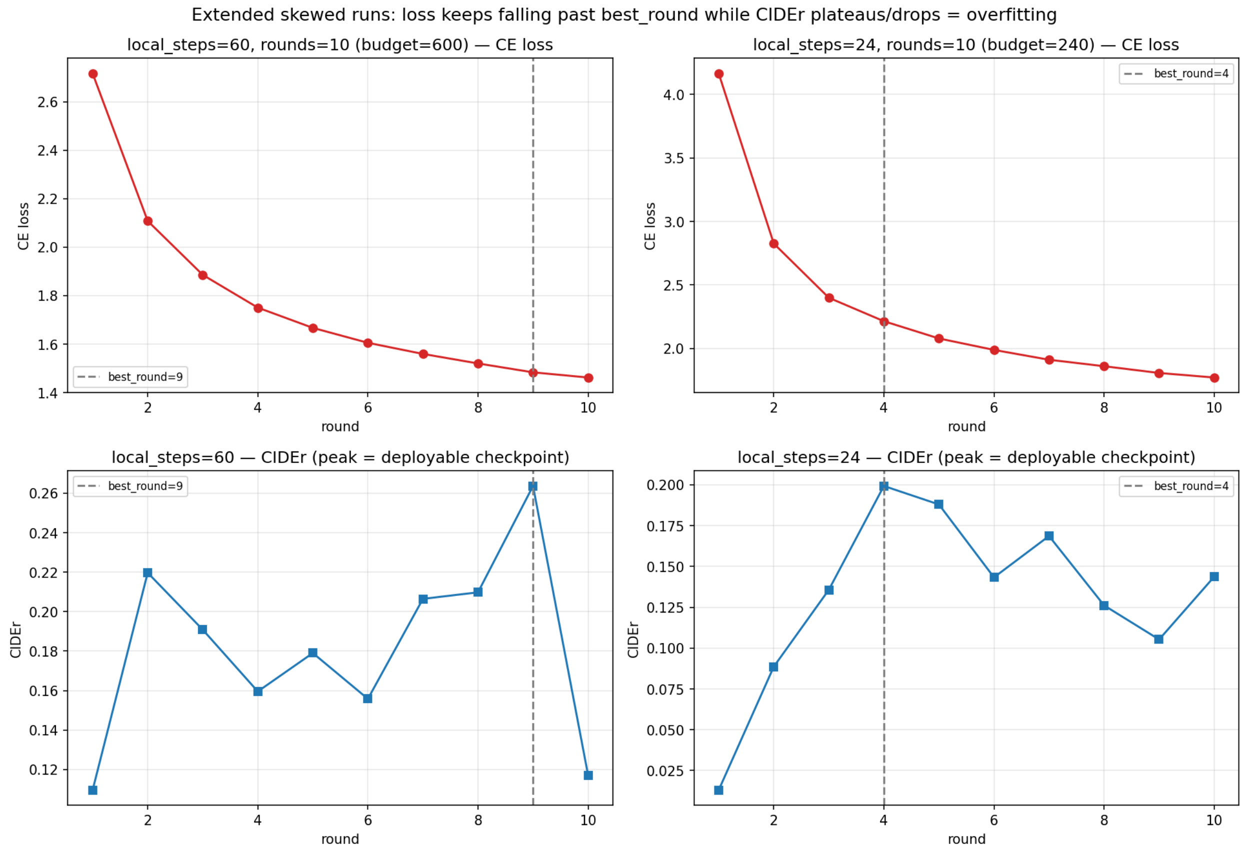
Green: best CIDEr reached at any point during the run (the checkpoint you'd actually deploy). Orange: CIDEr at the very last round. Red shading: the gap between them, i.e. how much quality was lost by training to the end instead of stopping at the peak.

Observation: Within the shared 120-step budget, the red shaded gap is generally small in both panels - the final round is usually close to the peak, though the gap widens somewhat at a few mid-range local_steps values (notably local_steps=5 in IID, where final CIDEr drops well below best CIDEr).

Inference: At this budget size, most runs are not yet deep into an overfitting regime by their last round - the model is generally still near its best when training stops. This sets up a useful contrast with Section 9, where runs deliberately trained beyond this budget show the gap opening up dramatically.

Practical takeaway: There's no strong, consistent relationship between local_steps and the size of this best-vs-final gap within the fixed budget - the risk of 'overshooting' the peak only becomes serious once training is extended well past this point, not within it.

9. Overfitting in the Extended Skewed Runs



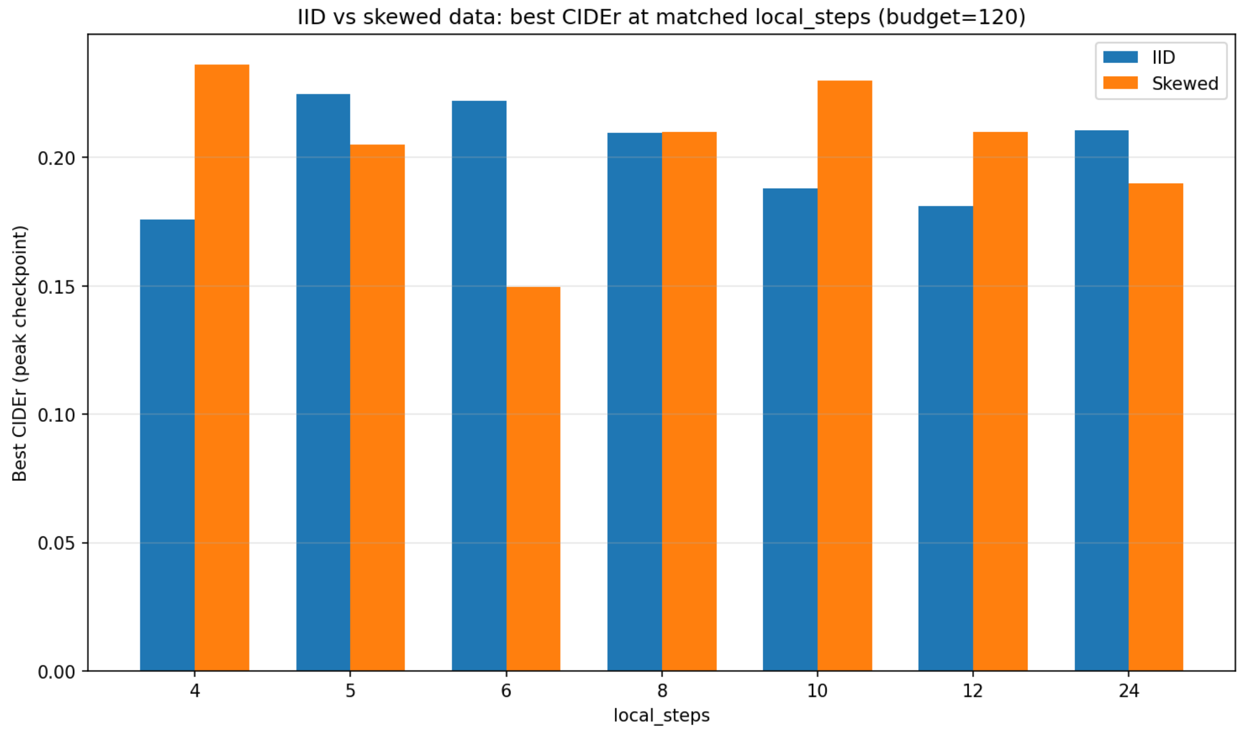
The two skewed runs deliberately trained past the shared 120-step budget: local_steps=60 (rounds=10, budget=600) and local_steps=24 (rounds=10, budget=240). Dashed line marks best_round (peak CIDEr).

Observation: In both runs, CE loss keeps falling smoothly all the way to the final round - it never signals a problem. CIDEr, however, peaks well before the end (round 9 of 10 for local_steps=60; round 4 of 10 for local_steps=24) and then drops sharply afterward. The local_steps=60 run is the most dramatic: CIDEr falls from a peak of 0.264 to just 0.118 in the single last round - a loss of more than half its value.

Inference: This is unambiguous, textbook overfitting, and it is only visible in the CIDEr curve, not the loss curve - loss actively improves while caption quality collapses. It confirms the pattern flagged in Section 1: loss and downstream generation quality decouple once a model is trained for too long relative to its data, and this decoupling is severe enough to erase most of the gains from the whole run if you are not checkpointing on the right metric.

Practical takeaway: Best-checkpoint selection must be based on CIDEr (or another task metric), never on loss, and training runs need frequent-enough evaluation to actually catch the peak before it passes - the local_steps=24 run's peak at round 4 of 10 would have been completely missed by anyone assuming quality keeps improving alongside loss for the whole run.

10. IID vs. Skewed: Best CIDEr at Matched local_steps



Grouped bars for the 7 local_steps values present in both distributions at the shared 120-step budget.

Observation: There is no consistent winner between distributions. local_steps=4 and local_steps=10 score meaningfully higher under skew than IID; local_steps=6 scores meaningfully lower under skew (0.150 vs. 0.222, the largest single gap in the chart); local_steps=8 is essentially tied.

Inference: This directly answers whether a single 'best' local_steps generalizes across data regimes: it does not. The same aggregation-frequency choice can help or hurt substantially depending on how skewed the client data is, which means local_steps is a hyperparameter that should be re-tuned per data regime rather than fixed once and reused.

Practical takeaway: If this system will eventually run on real, unknown client data whose skew is not known in advance, it would be safer to pick a local_steps value that performed reasonably (not necessarily best) in both distributions here - such as local_steps=8, which is roughly tied across both - over a value that only excels in one regime, such as local_steps=6 or local_steps=40.

Summary: The Local-Steps Trade-off

• **Higher local_steps is not simply 'better'.** It trades a higher potential peak CIDEr and smoother-looking curves against far fewer evaluation checkpoints to actually catch that peak, and - once training is extended - a faster and more severe collapse after the peak (Section 9). The measured client-divergence proxy (Section 7) did not cleanly show a downside here, but that measurement is confounded with training progress and should not be read as evidence that infrequent aggregation is divergence-free.

• **Loss is not a safe stopping signal at any local_steps value.** Across every section of this report, CE loss improves smoothly regardless of what CIDEr is doing, including while CIDEr is actively collapsing. Checkpoint selection must be driven by CIDEr (or another downstream metric).

• **Data skew changes the answer.** The same local_steps value can rank very differently under IID vs. skewed client data (Section 10) - e.g. local_steps=6 scores 0.222 under IID but only 0.150 under skew. Any 'optimal' aggregation frequency found on one data regime should not be assumed to transfer to another.

• **A moderate local_steps range (roughly 5-10) is the more robust operating point** in this data: it retains enough rounds to reliably monitor and stop at the true peak, and performs competitively (though not always best) in both the IID and skewed comparisons, rather than excelling in one regime and underperforming in the other.

• **Open question for follow-up:** whether infrequent aggregation genuinely increases client drift could not be cleanly answered from this data, because local_steps is entangled with 'how far into training the measurement lands' under the fixed-budget design. A cleaner experiment would fix rounds and vary local_steps (varying total budget), or sample client_loss_std at matched cumulative-step checkpoints.